❶ The method parameter is the binding model.

❷ A more complex example would do this work in an external service, in the application model.

❸ The result is exposed as a property and is used by the view to generate a response.

In chapters 6 and 14, you learned about routing and how it selects a Razor Page to execute. You can update the route template for the Razor Page to be `"CalculateSquare/ {number}"` by adding a `{number}` segment to the Razor Page's `@page` directive in the .cshtml file:

```
@page "{number}"
```

When a client requests the URL `/CalculateSquare/5`, the Razor Page framework uses routing to parse it for route parameters. This produces the route value pair

```
number=5
```

The Razor Page's `OnGet` page handler contains a single parameter—an integer called `number`—which is your binding model. When ASP.NET Core executes this page handler method, it will spot the expected parameter, flick through the route values associated with the request, and find the `number=5` pair. Then it can bind the `number` parameter to this route value and execute the method. The page handler method itself doesn't care where this value came from; it goes along its merry way, calculating the square of the value and setting it on the `Square` property.

The key thing to appreciate is that you didn't have to write any extra code to try to extract the `number` from the URL when the method executed. All you needed to do was create a method parameter (or public property) with the right name and let model binding do its magic.